

User Manual

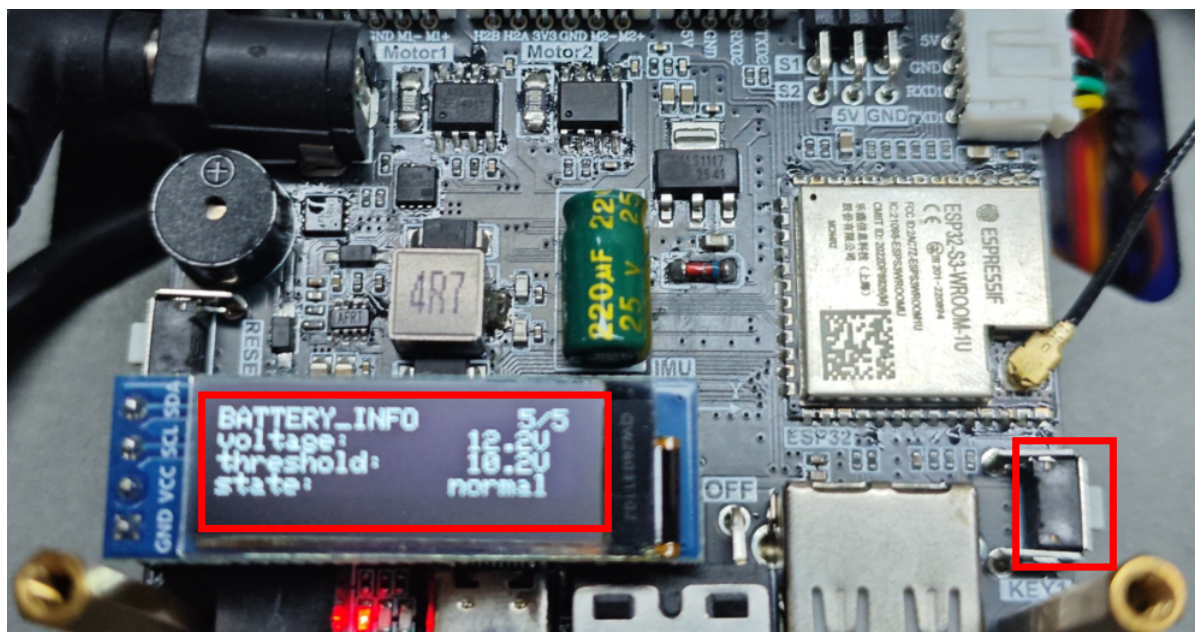
User Manual

1. Battery Charging Notes
 - 1.1 Low Battery Voltage Threshold Alarm
 - 1.2 Battery Charging
2. First-Time Usage Steps
 - 2.1 Assemble the Robot Car
 - 2.2 Download the Virtual Machine Image
 - 2.3 Configure the Virtual Machine to Connect to the Robot
3. Viewing the Source Code
 - 3.1 Viewing and Editing Source Code with Visual Studio Code
 - 3.2 Directory Structure Description
 - 3.3 Symbol Conventions
 - 3.4 How to Build ROS Packages

1. Battery Charging Notes

1.1 Low Battery Voltage Threshold Alarm

- When the battery voltage drops below the threshold (10.2V), the buzzer will emit a beep—beep—beep alarm. At this point you should charge the battery using the charger. Click the button on the right to switch to the **BATTREY_INFO** panel to view the current battery voltage and the voltage threshold.

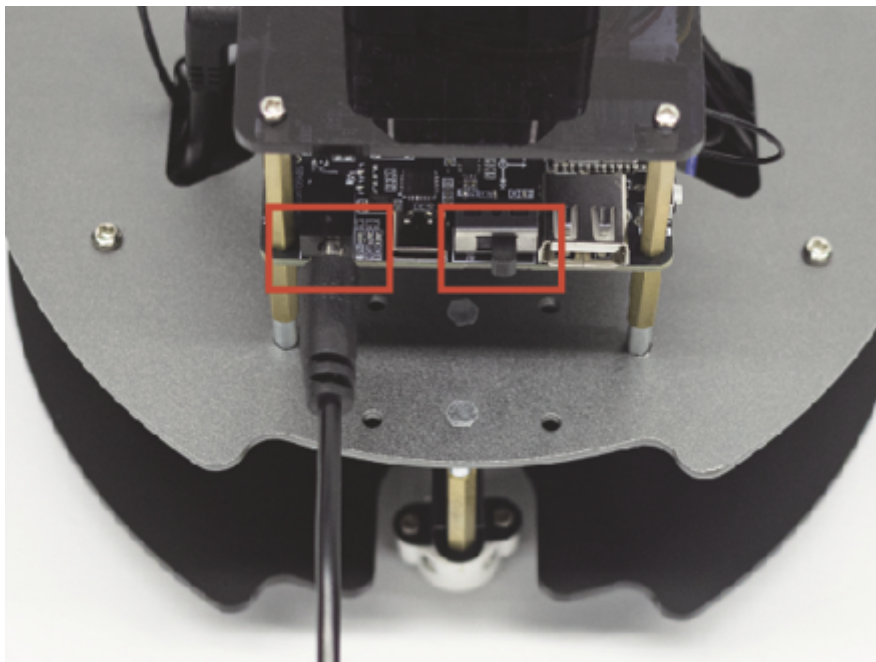


1.2 Battery Charging

- Connect the charger provided with the product to the power jack; the charger indicator light turns green



- Turn off the robot power, then connect the charger plug to the charging port on the controller board



- During charging, the charger indicator light is red; once the battery is fully charged, the indicator turns green



2. First-Time Usage Steps

- Follow the steps below to complete basic configuration such as robot assembly and virtual machine installation

2.1 Assemble the Robot Car

- Refer to [2. Assembly Tutorial] to complete the assembly of the robot car

2.2 Download the Virtual Machine Image

- Refer to the tutorial document [Preparation - Install VM and Start System] to download the virtual machine image provided with the product. The image file is located in the [Virtual Machine Image] folder of the supporting materials

2.3 Configure the Virtual Machine to Connect to the Robot

- Refer to the tutorial document [Preparation - Configure VM and Connect Robot], and use the RobotManager tool in the virtual machine to complete basic configuration such as the robot model, microROS agent address, and camera IP

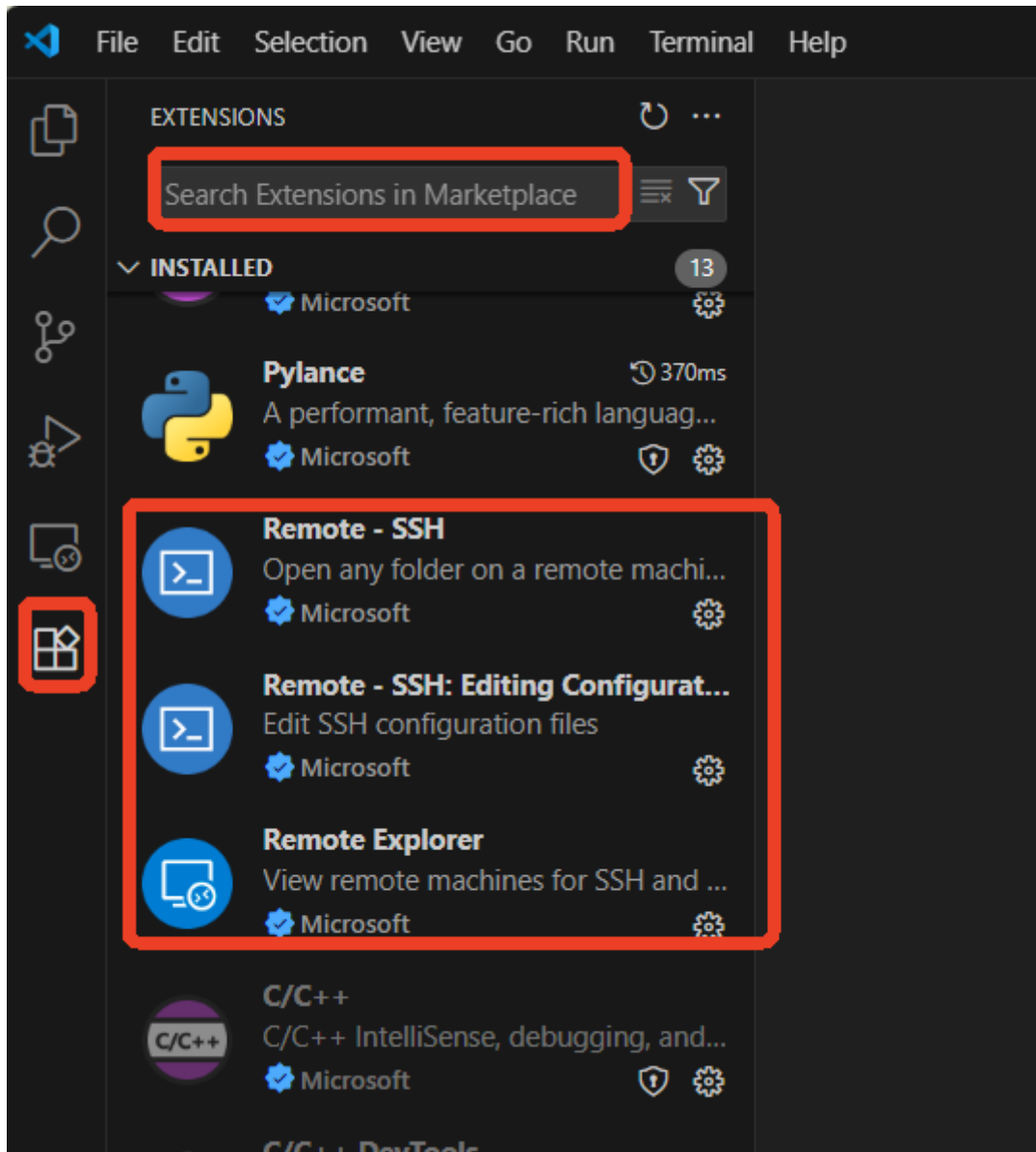
3. Viewing the Source Code

[!NOTE]

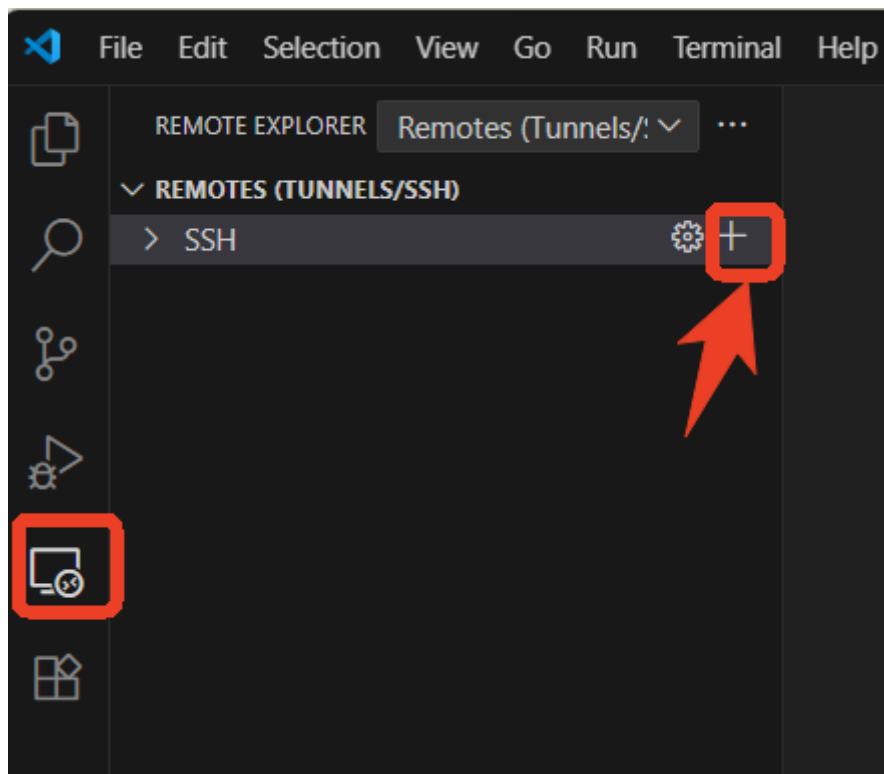
- Visual Studio Code is a lightweight, efficient, and extensible tool with integrated core development features at its heart, making it one of the most popular development tools in the world
- Official website: <https://code.visualstudio.com/>. If you have not installed Visual Studio Code, you can search for the official website and download and install it

3.1 Viewing and Editing Source Code with Visual Studio Code

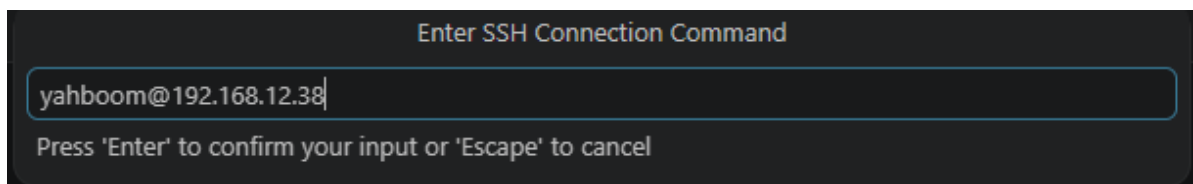
- Launch Visual Studio Code, click the Extensions icon on the left, and install three remote connection plugins: Remote-SSH, Remote - SSH: Editing Configuration Files, and Remote Explorer



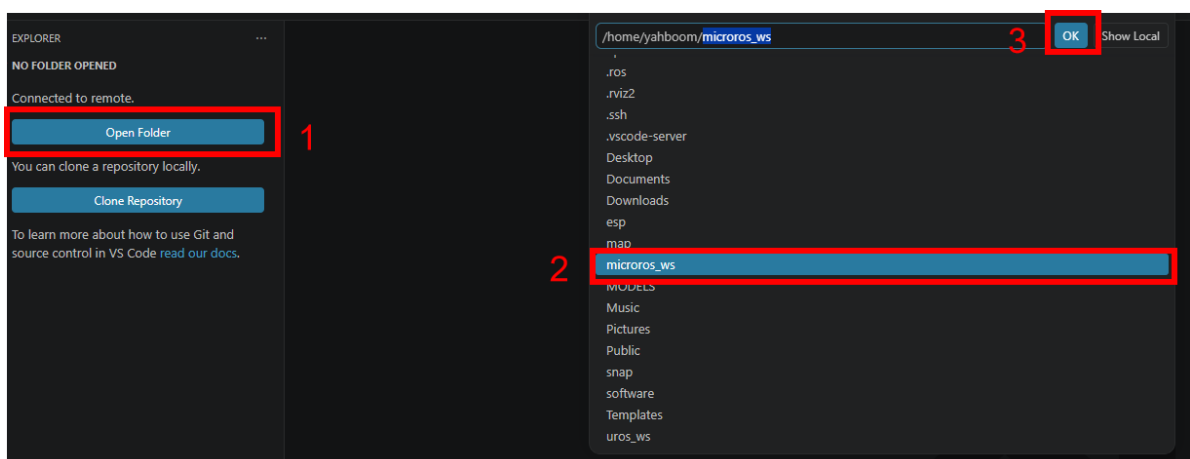
- Then click Remote Explorer on the left, and click New Remote



- In the pop-up input box, enter username@IP-address for your board, press Enter, then enter the user password yahboom to log in to the robot car



- Then click Open Folder; in the pop-up window you can select the folder you want to view. Here, select microros_ws as an example, then click OK



- The code workspace directory structure is as follows

3.2 Directory Structure Description

-  `src/` — Source code directory, containing the source code of all ROS 2 packages; it is the main development area of the entire workspace:

Package name	Function
<code>interfaces/</code>	Custom message/service/action interface definitions (msg, srv, action)
<code>microros_v2_bringup/</code>	Robot launch package: launch files, config files, rviz visualization configuration
<code>microros_v2_opencv_visual_pkg/</code>	OpenCV vision processing nodes
<code>microros_v2_opencv_base_pkg/</code>	OpenCV basic tutorial scripts (geometric transforms, image enhancement, drawing, etc.)
<code>microros_v2_yolo_pkg/</code>	YOLO object detection related nodes
<code>yolo_onnx_follow_pkg/</code>	Vision following based on the YOLO ONNX model
<code>microros_v2_kcf_follow_pkg/</code>	KCF target tracking and following
<code>microros_v2_mediapipe_pkg/</code>	MediaPipe human body/gesture perception features
<code>microros_v2_ptz_visual_control_pkg/</code>	Pan-tilt (PTZ) vision control
<code>microros_v2_no_ptz_visual_control_pkg/</code>	Vision control for the version without a pan-tilt
<code>multi_brains/</code> , <code>multi_brains_pre/</code>	Multi-"brain" decision/task planning modules (including LLM/Dify integration related configs)
<code>roadnet/</code>	Road network/navigation related (C++ package, containing graph data graphs)
<code>robot_description/</code>	Robot URDF model description
<code>robot-control/</code> , <code>ros_robot_app/</code>	Robot control and application layer
<code>common_demo/</code>	Common example code
<code>scripts/</code> , <code>robot_config/</code>	Helper scripts and robot configuration

-  `build/` — Build intermediate artifacts

Intermediate files generated during colcon compilation (CMake cache, object files, etc.), **can be deleted and regenerated by rebuilding**.

-  `install/` — Build installation output

The installation directory after colcon compilation, containing executables, libraries, Python packages, and launch files. After running `source install/setup.bash`, you can use the nodes and commands compiled here.

- `python_env/` — Python virtual environment

An independent Python environment managed by `uv`, to prevent vision-related dependencies (such as `onnxruntime`, `mediapipe`, etc.) from polluting the system environment:

- `microros_v2_env/` — main project Python environment, `yolo_env/` — YOLO dedicated environment
- `third_party_library/` — Independent workspace for third-party libraries

An independent colcon workspace dedicated to compiling and installing third-party SLAM-related libraries (`cartographer`, `cartographer_ros`, `slam_gmapping`, `openslam_gmapping`), isolated from the main workspace for easier maintenance.

- `cache/` — Runtime cache

Cache files generated during program runtime, such as debug images (`llm_camera.jpg`, `grid_images.png`), YOLO model inference verification images (`bus_predict_yolo11n/`), etc.

3.3 Symbol Conventions

- In the subsequent tutorial documents, the `$HOME` and `~` symbols represent the user home directory `/home/yahboom`. For example:

`$HOME/microros_ws/src/microros_v2_bringup/microros_v2_bringup/visual_track.py` is equivalent to `/home/yahboom/microros_ws/src/microros_v2_bringup/microros_v2_bringup/visual_track.py`

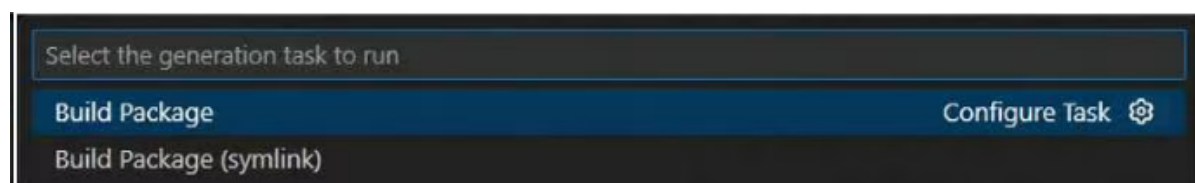
```
● yahboom@VM:~$ ls $HOME/microros_ws/src/microros_v2_bringup/microros_v2_bringup/visual_track.py
/home/yahboom/microros_ws/src/microros_v2_bringup/microros_v2_bringup/visual_track.py
● yahboom@VM:~$ cat $HOME/microros_ws/src/microros_v2_bringup/microros_v2_bringup/visual_track.py
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# author: Yahboom Technology ROS Robotics Team
# version: 202607

import time
import math
import numpy as np
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist, Vector3
from smart_tracker.msg import TrackData
```

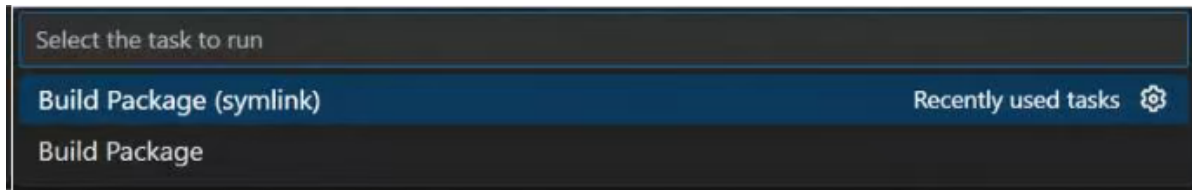
3.4 How to Build ROS Packages

Quick Method

- Press `ctrl+shift+B` in the Visual Studio Code interface, and a build selection panel will pop up. When building a normal package, select `Build Package`



- Select the package you want to build from the pop-up list
- When building `microros_v2_bringup` and `robot_description`, select `Build Package`. In subsequent use, modifications to the launch files and configuration in `microros_v2_bringup` will take effect directly, without needing to rebuild the package



Manual Build Method

```
cd ~/microros_ws/  
colcon build --packages-select <package_name>
```

- Example

```
yahboom@VM:~/microros_ws$ colcon build --packages-select multi_brains  
Starting >>> multi_brains  
Finished <<< multi_brains [0.59s]  
  
Summary: 1 package finished [0.77s]
```